



Universiti Malaysia Perlis

EMJ27204

Embedded System and Interfacing

Semester 2, 2025/2026

Mini Project Title:

Automated Parcel Storage and Collection System

SDG: SDG 9 (Industry, Innovation and Infrastructure)

Lecturers:

Assoc. Prof. Dr. Norasmadi Abdul Rahim

Dr. Ahmad Shakaff Ali Yeon

Group Members:

NO.	Name	Matric NO.
1.	Adel Husham Mohamedain Yousuf	231090057-1
2.	Abdalla Abuzaid Abdalla Abaker	241060086-1
3.	Malik Abuzaid Abdalla Abaker	241060087-1
4.	Yousif Osama Yousif Elnair	231060062-1
5.	Akram Yassir Elsir Ali	231060044-1

ABSTRACT

The rapid growth of online shopping has significantly increased the number of parcel deliveries within universities, making conventional manual parcel management increasingly inefficient, time-consuming, and prone to human error. This project presents the design and development of an **Automated Parcel Storage and Collection System** that integrates embedded systems, automation, and Internet of Things (IoT) technology to provide a secure and efficient parcel management solution. The system employs two Raspberry Pi Pico microcontrollers, where the first controller manages parcel delivery by validating student identification numbers, detecting parcel placement using a laser-LDR sensing system, and transporting parcels via a conveyor mechanism. The second controller manages parcel storage, generates secure four-digit collection PINs, controls parcel collection, and provides real-time communication through a Telegram Bot. Communication between the two controllers is achieved using one-way UART communication for parcel information transmission and a dedicated GPIO signal for monitoring storage availability. Experimental studies were conducted to determine the optimum operating parameters of the system. A threshold of **400 lux** was selected for reliable parcel detection based on statistical analysis of the laser-LDR measurements, while a conveyor running time of **15 seconds** was determined through repeated experimental measurements with an additional safety margin to ensure consistent parcel transportation. The developed system successfully automated the parcel delivery and collection process, provided secure PIN-based authentication, and delivered real-time Telegram notifications, including parcel arrival, collection reminders, confirmation messages, and security alerts. Overall, the proposed system demonstrated reliable performance, successfully achieved all project objectives, and provides a practical and scalable solution for modern parcel management within a university environment.

1.0 INTRODUCTION:

1.1 Background

The rapid growth of e-commerce and online shopping has significantly increased the number of parcel deliveries to residential areas, educational institutions, and workplaces. University students frequently receive parcels purchased through online shopping platforms, creating a greater demand for efficient parcel management systems.

Conventional parcel collection methods usually rely on manual handling, where parcels are received, recorded, stored, and distributed by administrative staff. As the number of delivered parcels increases, manual management becomes increasingly inefficient, resulting in longer waiting times, increased workload, and a higher possibility of human error.

One of the major challenges of traditional parcel collection systems is ensuring that parcels are collected only by their rightful owners. Manual verification methods are often time-consuming and may lead to unauthorized parcel collection or misplaced parcels. Furthermore, students are generally required to visit the collection office to check whether their parcels have arrived, as many conventional systems do not provide automatic notifications. This lack of real-time communication reduces user convenience and increases the administrative workload.

The advancement of embedded systems has enabled the development of automated solutions capable of improving both operational efficiency and system security. By integrating microcontrollers, sensors, actuators, wireless communication, and automation technologies, parcel management systems can perform delivery, storage, authentication, and notification processes with minimal human intervention. These technologies not only improve the accuracy of parcel handling but also enhance the overall user experience.

To address these challenges, this project presents an **Automated Parcel Storage and Collection System** developed using two Raspberry Pi Pico microcontrollers. The system automates the entire parcel handling process, beginning with parcel delivery and ending with secure parcel collection. The first microcontroller manages parcel delivery by verifying student identification, detecting parcel placement, and controlling the conveyor mechanism. The second microcontroller manages parcel storage, generates a unique collection PIN, provides Telegram notifications, controls parcel collection, and monitors system security. Communication between the two controllers enables the delivery and storage processes to operate efficiently while maintaining reliable system coordination.

The proposed system incorporates various hardware components, including matrix keypads, liquid crystal displays (LCDs), continuous rotation servo motors, stepper motors, laser and Light Dependent Resistor (LDR) sensors, LEDs, and a passive buzzer. In addition, a Telegram Bot is integrated to provide real-time parcel arrival notifications, collection reminders, collection confirmations, and security alerts. The combination of embedded hardware and software provides a secure, automated, and user-friendly solution for parcel management in a university environment.

1.2 Problem Statement

The increasing number of parcel deliveries resulting from the rapid growth of online shopping has created significant challenges in managing parcel collection within university environments. Most institutions still rely on conventional parcel management methods, where parcels are manually received, recorded, stored, and distributed by administrative staff. As the number of parcels continues to increase, manual management becomes less efficient and more prone to operational delays.

One of the primary issues associated with conventional parcel collection systems is the lack of a secure authentication mechanism. In many cases, parcel collection depends on manual verification by staff, which may result in unauthorized collection, misplaced parcels, or human errors during the identification process. This not only affects the security of the parcels but also reduces users' confidence in the collection system.

Another limitation is the absence of an automated notification service. Students often need to visit the collection office or wait for manual notifications to determine whether their parcels have arrived. This process is inconvenient, time-consuming, and may delay parcel collection, particularly when students are occupied with academic activities.

Furthermore, conventional parcel management requires continuous human involvement in recording deliveries, organizing storage locations, and assisting students during parcel collection. As the number of parcels increases, the administrative workload also increases, reducing the overall efficiency of the parcel management process.

Therefore, there is a need to develop an automated parcel storage and collection system capable of improving security, reducing manual intervention, and providing real-time notifications to users. By integrating embedded systems, secure PIN-based authentication, automated storage management, and wireless communication, the proposed system offers a more efficient, reliable, and user-friendly solution for parcel management within a university environment.

1.3 Objectives

The general objective of this project is to design and develop an **Automated Parcel Storage and Collection System** that provides a secure, efficient, and user-friendly solution for parcel management by integrating embedded systems, automation, and wireless communication technologies.

The specific objectives of this project are:

1. To design and develop an automated parcel delivery and collection system using two Raspberry Pi Pico microcontrollers to perform separate delivery and storage management tasks.
2. To implement secure parcel collection using randomly generated four-digit Personal Identification Numbers (PINs), ensuring that parcels can only be collected by their rightful recipients.
3. To establish reliable communication between the two Raspberry Pi Pico microcontrollers using one-way UART communication and a dedicated GPIO signal for storage status monitoring.
4. To integrate Telegram Bot services for real-time parcel arrival notifications, collection reminders, collection confirmations, and security alerts to improve user convenience and system monitoring.

1.4 Scope of Project

This project focuses on the design and development of an **Automated Parcel Storage and Collection System** intended for use in a university environment. The system is developed using two Raspberry Pi Pico microcontrollers, where each controller performs a dedicated function to achieve an efficient and secure parcel management process.

The first Raspberry Pi Pico (Pico 1) is responsible for the parcel delivery process. Its functions include validating the student's identification number entered through the matrix keypad, checking the availability of storage space, controlling the delivery gate using a continuous rotation servo motor, detecting the presence of a parcel using a laser and Light Dependent Resistor (LDR) sensor, operating the conveyor mechanism, and transmitting the student's identification number to the second microcontroller through one-way UART communication.

The second Raspberry Pi Pico W (Pico 2) is responsible for parcel storage and collection. It receives parcel information from Pico 1, generates a unique four-digit PIN for each

parcel, allocates an available storage location, controls the storage gates and collection doors, manages parcel collection through PIN verification, and monitors the overall storage status. Pico 2 also establishes a Wi-Fi connection to support Telegram Bot services, enabling parcel arrival notifications, collection reminders, parcel collection confirmations, and security alerts.

The communication between the two microcontrollers is limited to one-way UART transmission for parcel information and a dedicated GPIO signal for communicating storage availability. The system is designed to operate indoors under normal lighting conditions and supports a prototype consisting of three storage levels. The project does not include cloud database integration, mobile application development, or remote hardware control beyond the implemented Telegram notification service.

2.0 SYSTEM DESIGN AND IMPLEMENTATION

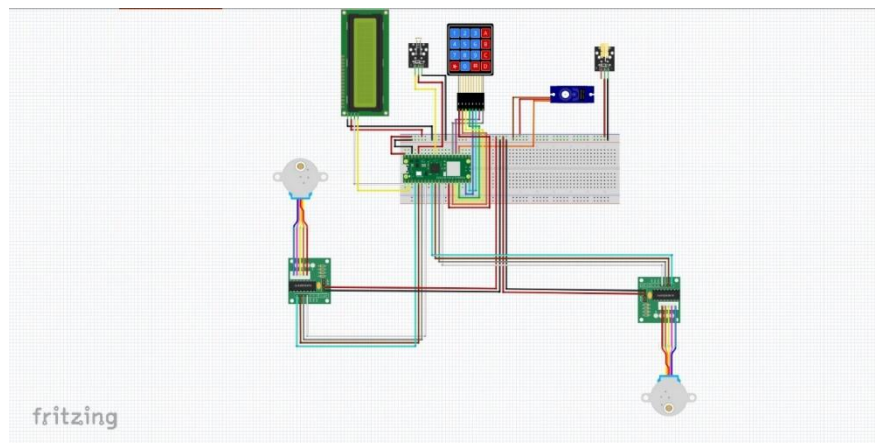
2.1 Overall System Architecture

The proposed Automated Parcel Storage and Collection System is designed using two Raspberry Pi Pico microcontrollers that operate cooperatively to perform different tasks within the parcel management process. Dividing the system into two independent controllers simplifies the overall system design by separating the delivery process from the storage and collection process while maintaining reliable communication between the two controllers.

2.2 Hardware Components & Design For Pico 1

Components	Quantity	Function
Raspberry Pi Pico	1	Main controller for the parcel delivery subsystem
Grove RGB LCD	1	Displays system status, instructions, and error messages
4×4 Matrix Keypad	1	Allows the delivery personnel to enter the student identification number
Continuous Rotation Servo Motor	1	Controls the opening and closing of the delivery gate
Laser Module	1	Emits a continuous light beam for parcel detection
Light Dependent Resistor Module (LDR)	1	Detects the interruption of the laser beam when a parcel is placed on the conveyor
28BYJ-48 Stepper Motor	2	Drives the conveyor system to transport parcels into the storage area
ULN2003 Driver Module	2	Drives the stepper motors from the Raspberry Pi Pico

Table 2.1



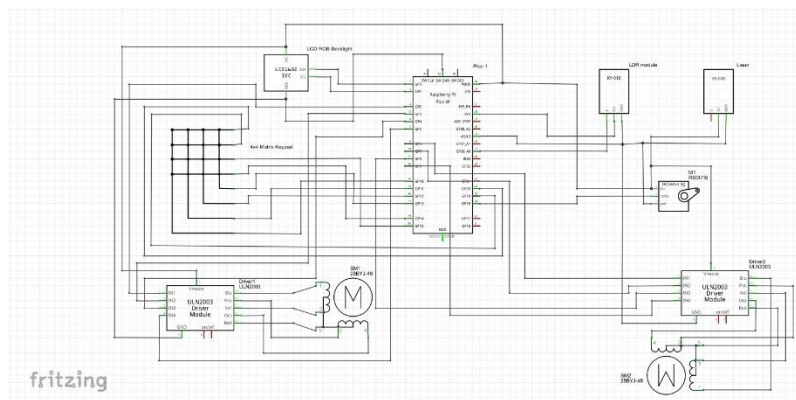


Figure 2.1: Pico 1 Schematic Diagram

Figure 2.1 illustrates the hardware schematic of the first controller (Pico 1), which is responsible for managing the parcel delivery process. Pico 1 controls the delivery gate, detects parcel placement, operates the conveyor mechanism, validates the student identification number, and communicates the parcel information to Pico 2 through UART.

2.3 Hardware Components & Design For Pico 2

Components	Quantity	Function
Raspberry Pi Pico	1	Main controller for the parcel storage and collection subsystem
16×2 I ² C LCD Display	1	Displays system status, PIN entry prompts, and parcel collection information
4×4 Matrix Keypad	1	Allows students and the administrator to enter PINs
Continuous Rotation Servo Motor	5	Controls the storage gates and parcel collection doors
Green LED	1	Indicates successful authentication and normal system operation
Red LED	1	Indicates incorrect PIN entry and alarm status
Passive Buzzer	1	Generates audible feedback and security alarm
330 Ω Resistor	2	Current-limiting resistors used to protect the green and red LEDs

Table 2.2

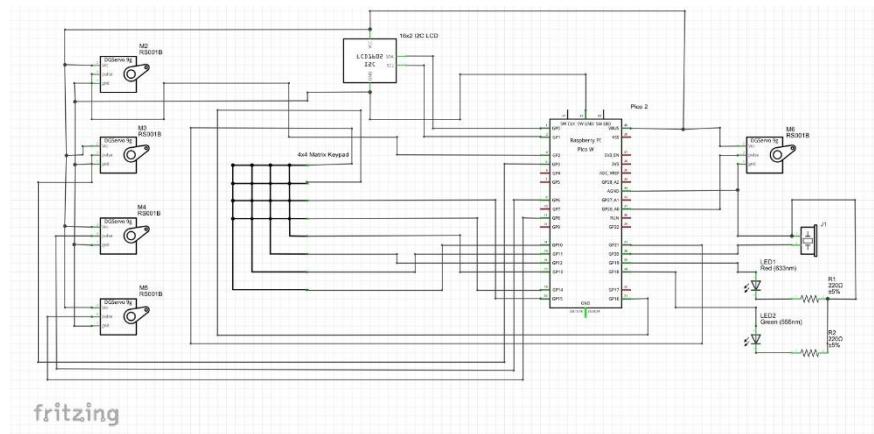
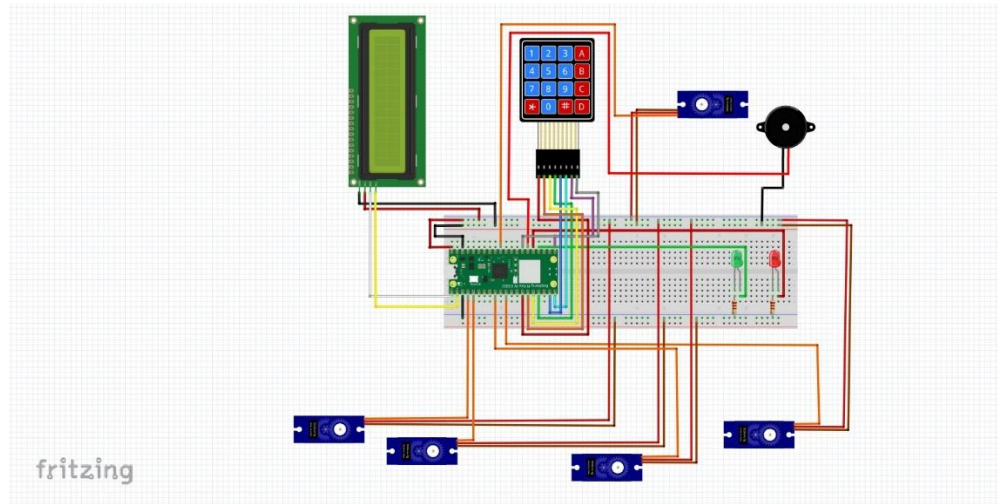
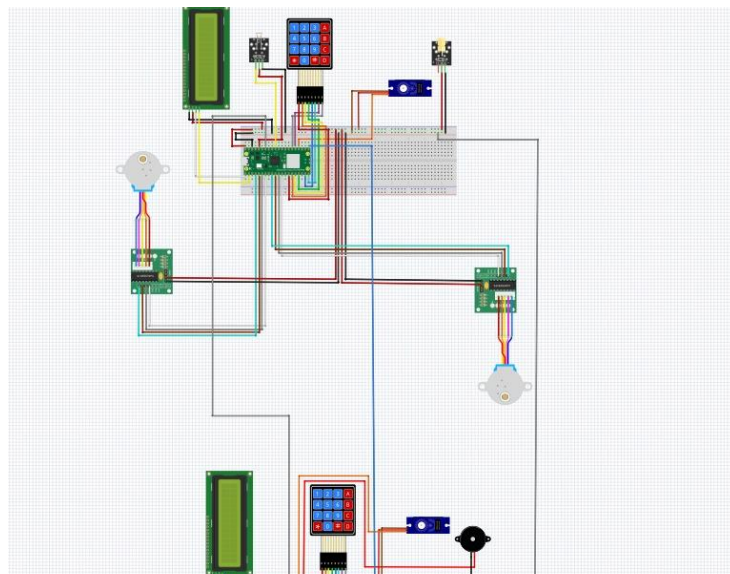


Figure 2.2: Pico 2 Schematic Diagram

Figure 2.2 illustrates the hardware schematic of the second controller (Pico 2), which is responsible for managing the parcel storage and collection process. Unlike Pico 1, which focuses on parcel delivery, Pico 2 manages parcel storage allocation, PIN generation, secure parcel collection, Telegram notifications, and system security.



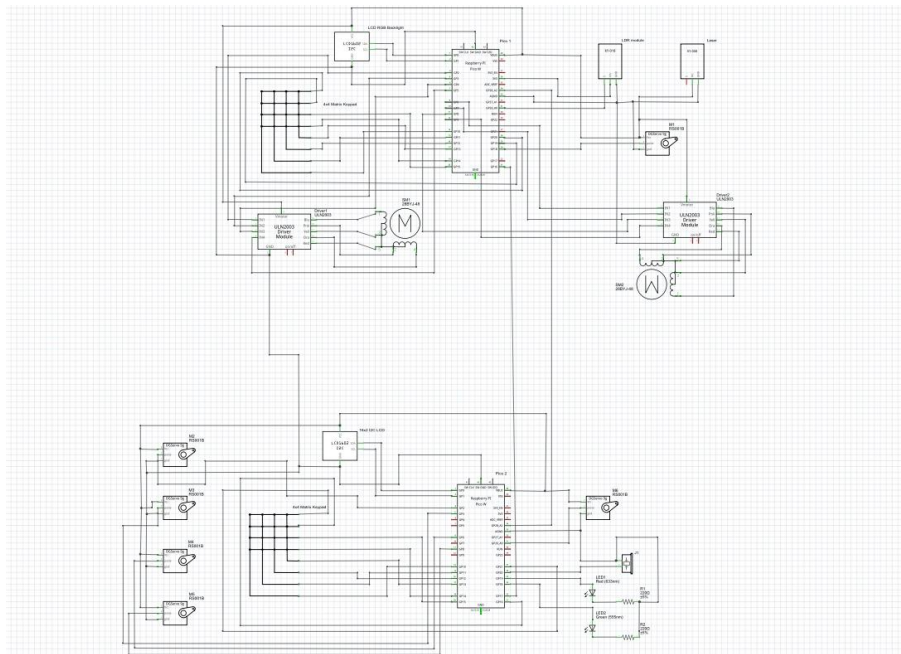


Figure 2.3: Complete Circuit Schematic Diagram

The integration of both controllers allows the overall system to be divided into two functional modules. The first module manages parcel acceptance and transportation, while the second module manages parcel storage, user authentication, notification services, and system security. This modular architecture improves system organization, simplifies software development, and allows each controller to operate independently without interfering with the tasks performed by the other controller.

2.4 Operational Flow

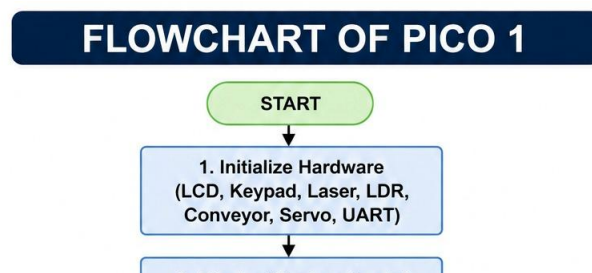


Figure 2.4: Pico 1 Flowchart

Figure 2.4 illustrates the operational flow of Pico 1, which is responsible for managing the parcel delivery process. The operation begins by initializing all hardware components, including the LCD display, matrix keypad, laser module, LDR sensor, delivery gate servo motor, conveyor system, and UART communication module. After initialization, the LCD

displays "**System Ready**", indicating that the system is ready to receive a student's identification number entered by the delivery personnel.

The entered identification number is then validated against the registered student database. If the identification number is invalid, the system displays an "**Invalid ID**" message and returns to the waiting state. If the identification number is valid, Pico 1 checks the storage availability signal received from Pico 2 through the dedicated GPIO communication line. When storage is available, the delivery gate opens for five seconds, allowing the parcel to be placed onto the conveyor. The laser-LDR sensing system continuously monitors the presence of the parcel, and once the measured light intensity falls below the predefined threshold of **400 lux**, the parcel is considered successfully detected.

The conveyor system is then activated for **15 seconds** to transport the parcel into the storage section. Upon completion of the conveyor operation, Pico 1 transmits the student's identification number to Pico 2 using one-way UART communication in the format "**STORE,StudentID**", indicating that a new parcel is ready for storage. Finally, the LCD displays "**Parcel Sent**", and the system returns to the initial waiting state, where it is ready to process the next parcel delivery.

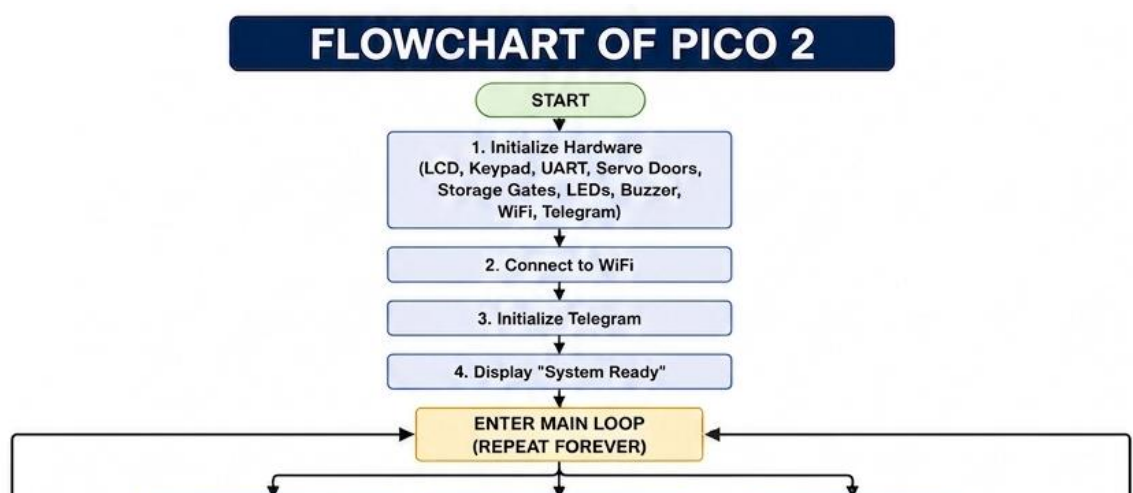


Figure 2.5: Pico 2 Flowchart

Figure 2.5 illustrates the operational flow of Pico 2, which is responsible for parcel storage, parcel collection, system security, and Telegram communication. The operation begins by initializing all hardware components, including the LCD display, matrix keypad, UART receiver, storage gate servos, collection door servos, LEDs, buzzer, Wi-Fi

module, and Telegram services. After successfully connecting to the wireless network and initializing the Telegram Bot, the LCD displays **"System Ready"**, indicating that the storage subsystem is ready for operation. The controller then enters a continuous main loop where it simultaneously updates the alarm status, monitors storage availability, reconnects to Wi-Fi when necessary, processes UART messages from Pico 1, validates user PIN entries, and checks incoming Telegram commands. When a UART message in the format **"STORE,StudentID"** is received, Pico 2 generates a random four-digit PIN and automatically stores the parcel by locating the first available storage position. If the upper storage locations are fully occupied, the existing parcels are shifted downward before assigning the parcel to the appropriate storage slot. After the parcel is successfully stored, Telegram notifications containing the generated PIN and storage information are sent to both the student and the system manager, while the storage status signal is updated and transmitted back to Pico 1 through the dedicated GPIO communication line. During parcel collection, the student enters the generated PIN through the keypad. If the system is in the alarm state, only the administrator PIN can deactivate the alarm and unlock the system. Otherwise, the entered PIN is validated against the stored parcel records. A valid PIN causes the corresponding collection door to open, displays a seven-second countdown on the LCD, removes the parcel from storage after collection, sends a Telegram confirmation message, and updates the storage availability. If an incorrect PIN is entered, the system increases the failed attempt counter. After three consecutive incorrect attempts, the security alarm is activated, causing the system to lock while the buzzer sounds continuously and both the red and green LEDs flash rapidly until the administrator successfully enters the correct administrator PIN. At the end of each operating cycle, Pico 2 checks the Telegram Bot for user commands such as **/start**, **/status**, and **/language**, while also monitoring the reminder timer before returning to the main loop to continue system operation.

2.5 Communication Between Pico 1 and Pico 2

Efficient communication between the two Raspberry Pi Pico microcontrollers is essential to ensure that the parcel delivery and storage processes operate as a single integrated system. Since the two controllers perform different responsibilities, an appropriate communication architecture was designed to allow information to be exchanged efficiently while minimizing hardware complexity.

The proposed system employs two independent communication methods, each serving a specific purpose. One-way Universal Asynchronous Receiver Transmitter (UART) communication is used to transmit parcel information from Pico 1 to Pico 2, while a

dedicated General-Purpose Input/Output (GPIO) signal is used to indicate the storage availability status from Pico 2 to Pico 1. By separating these two communication tasks, the system achieves reliable communication with a simple hardware and software implementation.

One-way UART communication is implemented to transfer parcel information from Pico 1 to Pico 2 after a parcel has been successfully delivered. Once the parcel is detected by the laser and LDR sensor and transported into the storage section by the conveyor, Pico 1 transmits the student's identification number to Pico 2 using a predefined UART message format.

The transmitted message consists of two fields separated by a comma, as shown below.

STORE,123

where:

- **STORE** represents the command instructing Pico 2 to store a new parcel.
- **123** represents the student identification number associated with the parcel.

Upon receiving the message, Pico 2 extracts the student identification number and performs all subsequent operations independently. These operations include generating a random four-digit PIN, allocating an available storage location, storing the parcel information, sending Telegram notifications, and preparing the parcel for future collection.

The communication is intentionally implemented as **one-way UART** because Pico 1 only needs to notify Pico 2 that a new parcel has arrived. After this information has been transmitted, Pico 2 completes the remaining processes without requiring any additional instructions or responses from Pico 1. As a result, bidirectional UART communication would increase both hardware and software complexity without providing any significant functional benefit.

The use of one-way UART therefore provides a simple, reliable, and efficient communication mechanism that satisfies the requirements of the proposed system.

Although UART is used to transfer parcel information, it is not used to communicate storage availability. Instead, the proposed system employs a dedicated GPIO signal to indicate whether storage space is available.

The GPIO output is controlled by Pico 2 according to the current storage occupancy maintained within the locker management system. Whenever a parcel is stored or

collected, Pico 2 automatically updates the storage database and determines whether additional storage locations remain available.

Two logic states are defined for the GPIO signal:

GPIO Output	Storage Status
HIGH (Logic 1)	Storage Available
LOW (Logic 0)	Storage Full

During every delivery request, Pico 1 reads this GPIO input before opening the delivery gate. If the GPIO signal is HIGH, the system proceeds with the delivery process. Conversely, if the GPIO signal is LOW, the system immediately displays a "Storage Full" message on the LCD and prevents further parcel deliveries until storage space becomes available.

Using a dedicated GPIO signal instead of UART provides a simple and efficient solution because the storage status consists of only two possible conditions: **available** or **full**. Transmitting this binary information through a single digital pin reduces communication overhead and allows Pico 1 to determine the storage status immediately without waiting for UART messages.

The communication process between the two Raspberry Pi Pico microcontrollers can be summarized as follows:

1. The delivery personnel enters a valid student identification number through the keypad connected to Pico 1.
2. Pico 1 reads the GPIO storage status signal provided by Pico 2.
3. If storage is available, Pico 1 opens the delivery gate and waits for parcel detection.
4. After the parcel is detected and transported by the conveyor, Pico 1 transmits the student's identification number to Pico 2 using one-way UART communication.
5. Pico 2 receives the UART message, generates a collection PIN, stores the parcel information, and sends the appropriate Telegram notifications.

6. Whenever a parcel is stored or collected, Pico 2 updates the storage database and adjusts the GPIO storage status signal accordingly.

3.0 SOFTWARE DESIGN AND IMPLEMENTATION

3.1 Object-Oriented Programming Structure

The software of the proposed **Automated Parcel Storage and Collection System** was developed using the **Object-Oriented Programming (OOP)** approach. Instead of implementing all functionalities in a single program, the software was divided into **18 independent Python modules**, each responsible for a specific hardware component or system function. This modular architecture improves software organization, code readability, debugging, maintenance, and future system expansion. The main program (code.py) of each Raspberry Pi Pico initializes the required modules and coordinates their execution throughout system operation.

Table 3.1 and **3.2** summarizes the object-oriented modules used in the project.

Module	Description
config.py	Stores GPIO assignments, system constants, and configuration parameters.
conveyor.py	Controls the conveyor mechanism using two stepper motors.
delivery.py	Implements the parcel delivery workflow, including student ID validation and parcel detection.
gate.py	Controls the delivery gate servo motor for parcel insertion.
laser.py	Detects parcel placement using the laser and LDR sensor.
lcd.py	Controls the Grove RGB LCD display for user messages and system status.
matrix_keypad.py	Reads the student identification number entered through the matrix keypad.
uart.py	Transmits parcel information to Pico 2 using one-way UART communication.

Table 3.1: OOP Modules Used in Pico 1

Module	Description
config.py	

	Stores GPIO assignments, Telegram settings, system constants, and configuration parameters.
buzzer.py	Controls the passive buzzer, LEDs, and security alarm functions.
lcd.py	Controls the 16×2 I ² C LCD display for parcel collection messages and system status.
locker.py	Manages parcel storage allocation, storage shifting, and storage status updates.
pickup.py	Handles parcel collection, PIN verification, and Telegram command processing.
servo.py	Controls the storage gates and parcel collection doors.
matrix_keypad.py	Reads PIN entries from students and the administrator.
telegram.py	Manages Telegram Bot communication, notifications, and user commands.
uart.py	Receives parcel information transmitted from Pico 1.
wifi_connect.py	Establishes and maintains the Wi-Fi connection required for Telegram communication.

Table 3.2: OOP Modules Used in Pico 2

complete source code for all object-oriented modules is included in **Appendix A**.

3.2 Main Program of Pico 1

The complete source code of the Pico 1 main program is provided below.

```

1  # =====
2  # SMART PARCEL SYSTEM
3  # PICO 1
4  # Main Program
5  # Version 1.0
6  # =====
7
8  import digitalio
9
10 # =====
11 # Configuration
12 # =====
13
14 from config import *
15
16 # =====
17 # Modules
18 # =====
19
20 from lcd import GroveLCD
21 from matrix_keypad import MatrixKeypad
22 from gate import Gate
23 from laser import LaserSensor
24 from conveyor import Conveyor
25 from uart import UARTSender
26 from delivery import DeliverySystem

```

```

27
28 # =====
29 # LCD
30 # =====
31
32 lcd = GroveLCD(
33
34     LCD_SCL,
35
36     LCD_SDA
37
38 )
39
40 # =====
41 # Matrix Keypad
42 # =
43
44 keypad = MatrixKeypad(
45
46     row_pins=KEYPAD_ROWS,
47
48     column_pins=KEYPAD_COLS,
49
50     valid_ids=VALID_IDS
51
52 )
53
54 # =====
55 # Delivery Gate
56 # =====
57
58 gate = Gate(
59
60     pin=GATE_PIN,
61
62     speed=GATE_SPEED,
63
64     move_time=GATE_MOVE_TIME
65
66 )
67
68 # =====
69 # Laser Sensor
70 # =====
71
72 laser = LaserSensor(
73
74     pin=LDR_PIN,
75
76     threshold=LASER_THRESHOLD
77
78 )
79
80 # =====
81 # Conveyor
82 # =====
83
84 conveyor = Conveyor(
85
86     motor1_pins=MOTOR1,
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120 # =====
121 # Delivery System
122 # =====
123
124 delivery = DeliverySystem(
125
126     lcd,
127
128     keypad,
129

```

```

130     gate,
131
132     laser,
133
134     conveyor,
135
136     uart,
137
138     storage_ready
139
140 )
141
142 # =====
143 # System Startup
144 # =====

```

```

145
146 print("=====")
147 print("      SMART PARCEL SYSTEM V1.0")
148 print("      PICO 1")
149 print("=====")
150
151 lcd.ready()
152
153 print("System Ready...")
154
155 # =====
156 # Main Loop
157 # =====
158
159 while True:
160
161     delivery.process()

```

3.3 Main Program of Pico 2

The complete source code of the Pico 2 main program is provided below.

```

1  # =====
2  # SMART PARCEL SYSTEM
3  # PICO 2 W
4  # Main Program
5  # Version 2.0
6  # =====
7
8  import digitalio
9
10 from config import *
11
12 from lcd import LCD
13 from matrix_keypad import MatrixKeypad
14 from servo import ContinuousServo
15 from buzzer import SecuritySystem
16 from locker import Locker
17 from pickup import PickupSystem
18 from uart import UARTReceiver
19
20 from wifi_connect import WiFiConnect
21 from telegram import TelegramBot
22
23
24 # =====
25 # LCD
26 # =====
27
28 lcd = LCD(
29     LCD_SCL,
30     LCD_SDA,
31     LCD_ADDRESS
32 )
33
34 lcd.custom(
35     "SMART PARCEL",
36     "Starting..."
37 )
38
39
40
41
42
43

```

```

44 )
45
46 # =====
47 # Matrix Keypad
48 # =====
49
50 keypad = MatrixKeypad(
51     row_pins=KEYPAD_ROWS,
52     column_pins=KEYPAD_COLS
53 )
54
55 # =====
56 # UART
57 # =====
58
59 uart = UARTReceiver(
60     UART_RX
61 )
62
63 # =====
64 # Security System
65 # =====
66
67 security = SecuritySystem(
68     GREEN_LED_PIN,
69     RED_LED_PIN,
70     BUZZER_PIN,
71     BUZZER_FREQUENCY
72 )
73
74 # =====
75 # Floor Gates
76 # =====

```

```

87
88 third_gate = ContinuousServo(
89     THIRD_GATE_PIN,
90     THIRD_GATE_SPEED,
91     SERVO_MOVE_TIME
92 )
93
94 second_gate = ContinuousServo(
95     SECOND_GATE_PIN,
96     SECOND_GATE_SPEED,
97     SERVO_MOVE_TIME
98 )
99
100 # =====
101 # Collection Doors
102 # =====
103
104 ground_door = ContinuousServo(
105     GROUND_DOOR_PIN,
106 )
107
108 second_door = ContinuousServo(
109     SECOND_DOOR_PIN,
110     SECOND_DOOR_SPEED,
111     SERVO_MOVE_TIME
112 )
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129

```

```

130 )
131
132 third_door = ContinuousServo(
133     THIRD_DOOR_PIN,
134     THIRD_DOOR_SPEED,
135     SERVO_MOVE_TIME
136 )
137
138 # =====
139 # Door Dictionary
140 # =====
141
142 doors = {
143     "GROUND": ground_door,
144     "SECOND": second_door,
145     "THIRD": third_door
146 }
147
148 # =====
149 # WiFi
150 # =====
151
152 lcd.custom(
153     "Connecting",
154     "WiFi..."
155 )
156
157 wifi = WifiConnect(
158     WIFI_SSID,
159     WIFI_PASSWORD
160 )

```

```

173 )
174
175 wifi.connect()
176
177 # =====
178 # Telegram
179 # =====
180
181 telegram = TelegramBot(
182     BOT_TOKEN,
183     wifi.get_requests()
184 )
185
186 # =====
187 # Locker
188 # =====
189
190
191
192
193

```

```

196     lcd,
197     third_gate,
198     second_gate,
199     telegram
200 )
201 # =====
202 # Pickup System
203 # =====
204 pickup = PickupSystem(
205     lcd=lcd,
206     keypad=keypad,
207 )

```

```

216     security=security,
217     locker=locker,
218     doors=doors,
219     admin_pin=ADMIN_PIN,
220     telegram=telegram,
221     door_open_time=DOOR_OPEN_TIME
222 )
223 telegram.pickup = pickup
224 # =====
225 # Storage Status Output
226 # =====
227 storage_status = digitalio.DigitalInOut(
228     STORAGE_STATUS_PIN
229 )
230 storage_status.direction = digitalio.Direction.OUTPUT
231 storage_status.value = True
232 # =====
233 # Startup
234 # =====
235 print("=====")
236 print(" SMART PARCEL SYSTEM")
237 print(" PICO 2 W")
238 print("=====")
239 print("WiFi Connected")
240 print("Telegram Ready")
241 print("System Ready")
242 lcd.ready()
243 # =====
244 # Main Loop
245 # =====

```

```

266
267 while True:
268
269     # -----
270     # Alarm Update
271     # -----
272
273     security.update()
274
275     # -----
276     # Storage Status
277     # -----
278
279     storage_status.value = not locker.is_full()
280
281     # -----
282     # WiFi Reconnect
283     # -----
284
285     wifi.reconnect()
286
287     telegram.set_requests(
288         wifi.get_requests()
289     )
290
291     # -----
292     # UART Receive
293     # -----
294
295     message = uart.receive()
296
297     if message:
298
299         print("UART :", message)
300
301

```

```

302
303     parts = message.split(",")
304
305     if len(parts) == 2:
306
307         command = parts[0]
308
309         student = parts[1]
310
311         if command == "STORE":
312
313             pickup.receive(student)
314
315         else:
316
317             print("Invalid UART Message")
318
319     # -----
320     # Student Keypad
321     # -----
322
323     pickup.check_pin()
324
325     # -----
326     # Telegram
327     # -----
328
329     pickup.check_telegram()

```

4.0 EXPERIMENTAL DESIGN AND PARAMETER SELECTION

4.1 Determination of the LDR Detection Threshold

Reliable parcel detection is one of the most important functions of the Automated Parcel Storage and Collection System. The system uses a laser module and a Light Dependent Resistor (LDR) to detect the presence of a parcel on the conveyor. Under normal operating conditions, the laser continuously illuminates the LDR sensor, producing a high light intensity reading. When a parcel is placed on the conveyor, it interrupts the laser beam, causing the measured light intensity to decrease significantly. Therefore, it was necessary to determine an appropriate threshold value that could accurately distinguish between the presence and absence of a parcel while minimizing false detections caused by variations in ambient lighting.

To determine the optimum threshold, several experiments were conducted under normal indoor lighting conditions. During each experiment, the laser was aligned directly with the LDR sensor, and the measured light intensity was recorded both when the laser beam reached the LDR directly and when the beam was blocked by a parcel. The measurements were repeated several times to ensure consistency and reliability of the recorded data.

The experimental procedure consisted of the following steps:

1. The laser module was aligned directly with the LDR sensor.
2. The ambient lighting condition was kept constant throughout the experiment.
3. The lux value was recorded while the laser beam directly illuminated the LDR sensor.
4. A parcel was placed between the laser module and the LDR sensor.
5. The lux value was recorded again after the laser beam was interrupted.
6. Steps 3 to 5 were repeated for multiple trials.
7. The recorded measurements were analysed to determine an appropriate detection threshold.

The experimental results are presented in **Table 4.1**, while **Figure 4.1** illustrates the corresponding lux measurements obtained during the experiment.

Trial	Laser Reaches LDR (Lux)	Parcel Blocks Laser (Lux)
1	816	238
2	809	221
3	823	247
4	814	231
5	806	216
6	820	242
7	812	228
8	827	235
9	818	223
10	811	239

Table 4.1: Experimental Lux Measurements

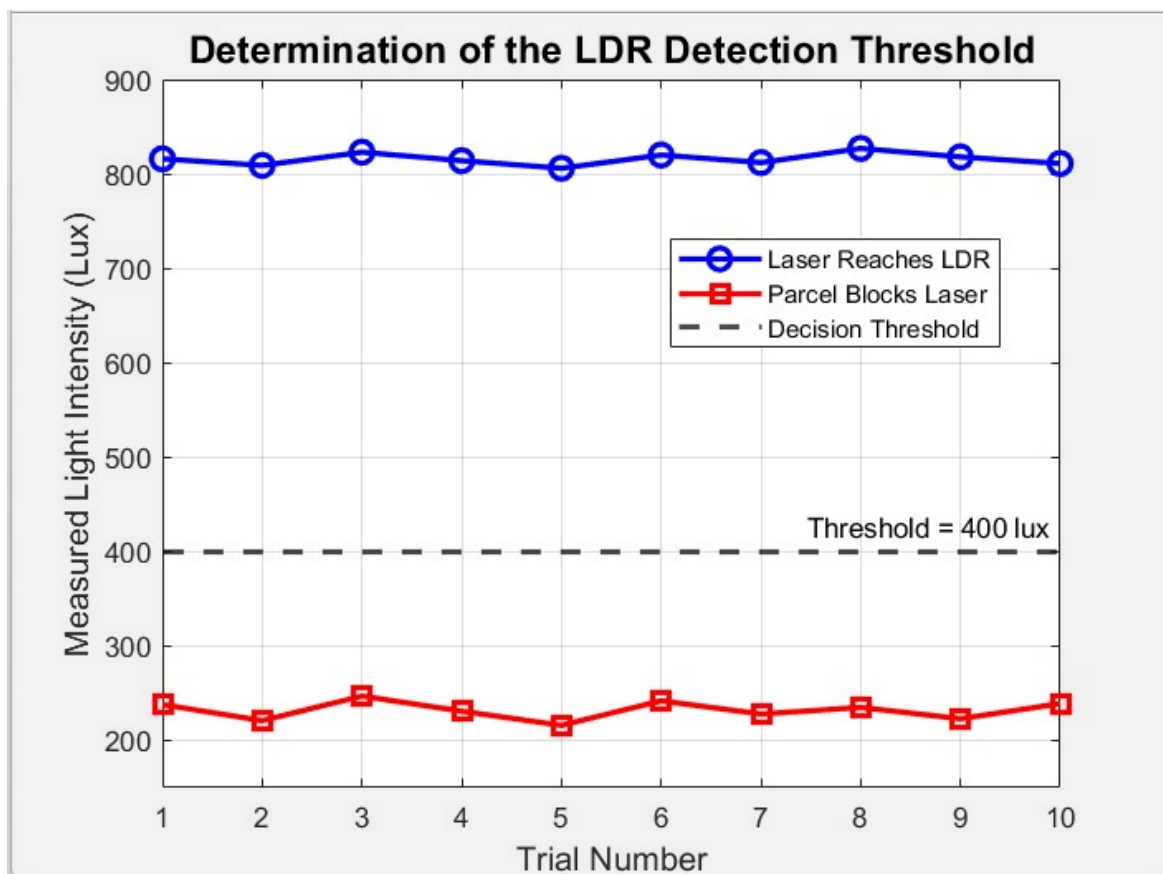


Figure 4.1: Comparison of Lux Measurements During Parcel Detection

Statistical Summary

Laser ON

Mean:

$$\mu_1 = \frac{816 + 809 + \dots + 811}{10} = 815.6 \text{ lux}$$

Minimum = **806 lux**

Maximum = **827 lux**

Standard Deviation $\approx$ **6.6 lux**

Parcel Blocking Laser

Mean:

$$\mu_2 = \frac{238 + 221 + \dots + 239}{10} = 232.0 \text{ lux}$$

Minimum = **216 lux**

Maximum = **247 lux**

Standard Deviation $\approx$ **10.0 lux**

The statistical analysis indicates that the measured light intensity remained consistently high when the laser beam directly illuminated the LDR sensor, with an average value of **815.6 lux** and a relatively small standard deviation of **6.6 lux**, demonstrating stable sensor performance under normal operating conditions. When the laser beam was interrupted by a parcel, the measured light intensity decreased significantly to an average of **232.0 lux**, with a standard deviation of **10.0 lux**. Although slight variations were observed in both operating conditions due to minor changes in sensor response and ambient lighting, the measurements remained highly consistent throughout the experiment.

Furthermore, the maximum recorded lux value during parcel detection (**247 lux**) was substantially lower than the minimum lux value recorded when the laser directly illuminated the LDR (**806 lux**). This resulted in a clear separation of approximately **559 lux** between the two operating conditions, indicating that the laser-LDR sensing system can reliably distinguish between the presence and absence of a parcel.

Engineering Decision

Based on the experimental results and statistical analysis, the operating threshold for parcel detection was determined using a **threshold-based decision approach**. Since the measured lux values corresponding to the two operating conditions formed two completely separated groups with no overlapping measurements, any threshold value between **247 lux** and **806 lux** would theoretically classify all observations correctly.

However, rather than selecting a threshold close to either operating region, a threshold value of **400 lux** was selected to provide a large safety margin between the two measurement clusters. This safety margin improves the robustness of the detection system by reducing the possibility of incorrect parcel detection caused by sensor noise, fluctuations in ambient lighting, slight laser misalignment, electrical interference, or long-term sensor ageing.

Consequently, whenever the measured light intensity falls **below 400 lux**, the system interprets the event as successful parcel detection. Conversely, lux values **greater than or equal to 400 lux** indicate that the laser beam is still reaching the LDR sensor and no parcel is present. Experimental testing demonstrated that this threshold provided stable and reliable parcel detection throughout the development of the Automated Parcel Storage and Collection System.

4.2 Determination of Conveyor Running Time

The conveyor system is responsible for transporting the parcel from the delivery section to the storage compartment. To ensure reliable parcel transportation, it was necessary to determine an appropriate conveyor running time that would allow every parcel to reach the storage area completely before the conveyor stopped. Since the transportation time may vary slightly due to differences in parcel position, friction, conveyor loading, and minor mechanical variations, an experimental study was conducted to determine the average transportation time under normal operating conditions.

The experiment was performed by placing the parcel at the starting position of the conveyor and measuring the time required for the parcel to reach the storage compartment using a stopwatch. The experiment was repeated for multiple trials under identical operating conditions to improve the reliability of the measurements. The recorded transportation times were then analysed to determine the average time required for successful parcel delivery.

The experimental procedure consisted of the following steps:

1. Place the parcel at the starting position of the conveyor.
2. Start the conveyor and simultaneously start the stopwatch.
3. Stop the stopwatch immediately after the parcel reaches the storage compartment.
4. Record the measured transportation time.
5. Repeat the experiment for multiple trials under the same operating conditions.
6. Calculate the average transportation time.
7. Add an additional safety margin to compensate for possible variations during actual system operation.
8. Implement the final conveyor running time in the system.

The experimental measurements are presented in **Table 4.2**, while **Figure 4.2** illustrates the measured transportation time for each experimental trial.

Trial	Transportation Time (seconds)
1	13.6
2	13.9
3	14.1
4	13.8
5	14.0
6	13.7
7	14.2
8	13.9
9	14.1
10	13.8

Table 4.2: Experimental Conveyor Transportation Time

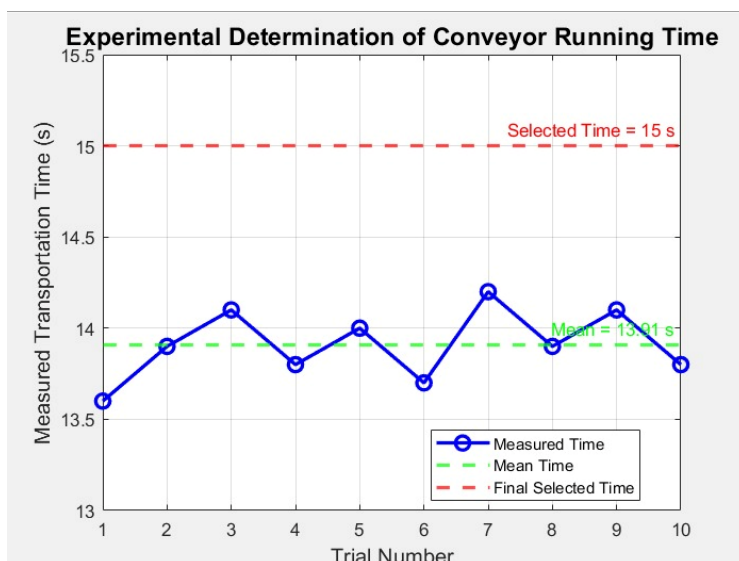


Figure 4.2: Measured Conveyor Transportation Time for Each Experimental Trial

Statistical Summary

Statistical Parameter	Value
Number of Samples	10
Mean	13.91 s
Minimum	13.6 s
Maximum	14.2 s
Range	0.60 s
Standard Deviation	0.20 s

Engineering Decision

The experimental results indicate that the parcel required an average transportation time of approximately **13.91 seconds** to reach the storage compartment. The relatively small standard deviation of **0.20 seconds** demonstrates that the conveyor system operated consistently throughout the experiment, with only minor variations between successive trials.

Although the measured average transportation time was approximately **14 seconds**, the final conveyor running time implemented in the Automated Parcel Storage and Collection System was increased to **15 seconds**. This additional one-second safety margin was intentionally introduced to compensate for practical operating conditions that may not be present during laboratory testing, such as slight differences in parcel dimensions, variations in parcel weight, increased mechanical friction, minor conveyor speed fluctuations, or temporary obstructions during parcel transportation.

By incorporating this safety margin, the system ensures that every parcel reaches the storage compartment completely before the conveyor stops, thereby improving the overall reliability of the parcel delivery process. Since the additional one second has a negligible effect on the total delivery cycle but significantly increases operational robustness, a conveyor running time of **15 seconds** was selected as the final operating parameter.

5.0 IOT IMPLEMENTATION

5.1 Proposed Structure

The proposed Automated Parcel Storage and Collection System incorporates Internet of Things (IoT) technology through the integration of a Telegram Bot. The Raspberry Pi Pico W establishes a wireless connection to the local Wi-Fi network, allowing the system to communicate with Telegram servers and exchange information with registered users in real time. This IoT functionality enables students and the parcel manager to receive immediate notifications regarding parcel status without requiring physical interaction with the system.

5.2 Telegram Bot Results

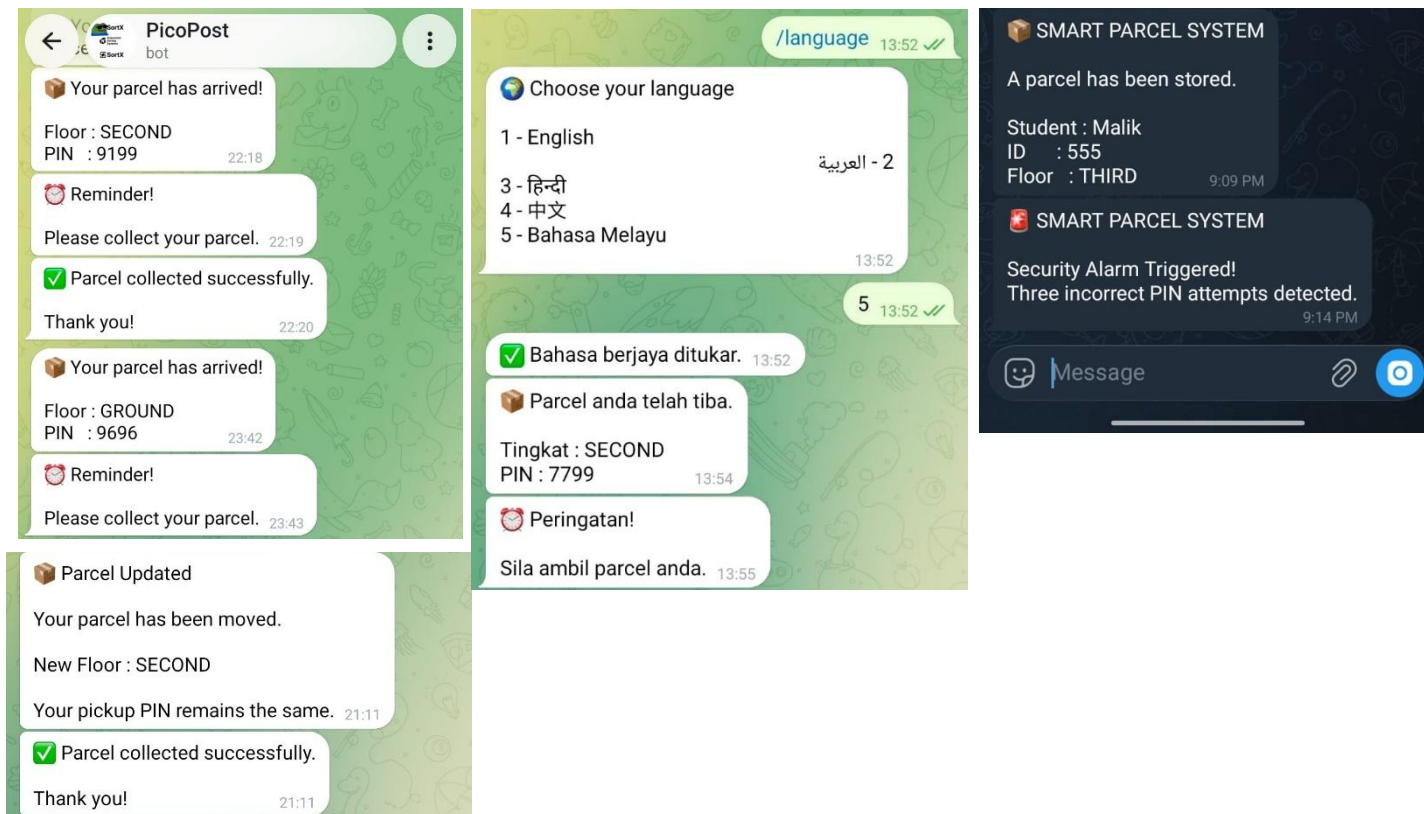


Figure 5.1 Telegram Bot Interfaces Demonstrating the IoT Functionality of the Automated Parcel Storage and Collection System

Figure 5.1 presents the Telegram Bot interfaces implemented in the proposed Automated Parcel Storage and Collection System. The Telegram Bot serves as the primary Internet of Things (IoT) communication platform between the system, students, and the parcel manager. Through this interface, users can interact with the system using commands such as **/start**, **/status**, and **/language**, while receiving real-time notifications regarding parcel activities. After a parcel is successfully stored, the system automatically sends the student a parcel arrival notification containing the allocated storage location and the randomly generated four-digit collection PIN. If the parcel is not collected within the predefined period, a reminder notification is issued to encourage timely collection. Upon successful parcel collection, a confirmation message is sent to the student, while the parcel manager receives notifications whenever a parcel is stored, the storage becomes full, or the security alarm is activated following three consecutive incorrect PIN attempts. In addition, the Telegram Bot supports multiple languages, allowing users to receive notifications in their preferred language. These results demonstrate the successful integration of IoT technology into the proposed system, providing reliable real-time communication, improved user convenience, and enhanced monitoring capabilities.

6.0 RESULTS AND DISCUSSION

The experimental results demonstrate that the proposed **Automated Parcel Storage and Collection System** successfully achieved all the objectives defined at the beginning of the project. The two Raspberry Pi Pico microcontrollers communicated reliably through one-way UART communication, while the dedicated GPIO signal accurately indicated the storage availability status between the two controllers. Throughout system testing, the communication architecture operated consistently without data transmission errors.

The parcel delivery and collection processes were successfully automated. Pico 1 accurately verified student identification numbers, detected parcel placement using the laser-LDR sensing system, and transported parcels to the storage section using the conveyor mechanism. After receiving the parcel information, Pico 2 successfully generated unique four-digit PINs, managed parcel storage, and authenticated students during parcel collection. The implemented security mechanism effectively activated the alarm after three consecutive incorrect PIN attempts and could only be reset using the administrator PIN.

The integration of the Telegram Bot further enhanced the functionality of the system by providing real-time notifications to both students and the parcel manager. The successful

implementation of parcel arrival notifications, collection reminders, collection confirmations, and security alerts demonstrated the effectiveness of the IoT features in improving communication and user convenience.

Overall, the developed system operated reliably under normal operating conditions and demonstrated that embedded systems, wireless communication, and secure authentication can be effectively integrated to automate parcel storage and collection within a university environment.

7.0 ADVANTAGES AND LIMITATIONS

7.1 Advantages

The developed Automated Parcel Storage and Collection System offers several advantages over conventional manual parcel management systems.

Firstly, the system automates the entire parcel handling process, from parcel delivery to collection, thereby reducing the need for continuous human intervention. This improves operational efficiency and minimizes the possibility of human error during parcel management.

Secondly, the implementation of randomly generated four-digit PINs enhances system security by ensuring that parcels can only be collected by their rightful recipients. The additional security alarm activated after three consecutive incorrect PIN attempts further protects the stored parcels against unauthorized access.

Another significant advantage is the integration of Internet of Things (IoT) technology through the Telegram Bot. Real-time notifications allow students to receive parcel arrival messages, collection reminders, and confirmation messages, while the parcel manager is immediately informed about storage activities and security alerts. This improves communication and enhances the overall user experience.

Furthermore, the modular architecture based on two Raspberry Pi Pico microcontrollers simplifies both the hardware and software design. Separating the parcel delivery process from the parcel storage and collection process improves system organization, reduces processing complexity, and facilitates future maintenance and system expansion.

7.2 Limitations

Despite the successful implementation of the proposed system, several limitations were identified during the development and evaluation of the project.

Firstly, the prototype supports only **three storage compartments**, limiting the number of parcels that can be stored simultaneously. Expanding the storage capacity would require additional hardware components and modifications to the storage management algorithm.

Secondly, the current storage mechanism depends entirely on students collecting their parcels. If a student fails to collect a parcel, that parcel continues occupying its assigned storage position, preventing the automatic shifting mechanism from freeing additional storage space. Consequently, new parcel deliveries may eventually be rejected once all storage compartments become occupied. The current prototype does not provide an administrator access mechanism that allows stored parcels to be removed manually in such situations.

Thirdly, the system depends on a stable Wi-Fi connection for Telegram communication. Any interruption to the wireless network prevents the transmission of real-time notifications, although the parcel storage and collection functions continue to operate normally.

Finally, the software operates using a conventional sequential execution approach. Although this implementation satisfies the functional requirements of the prototype, it does not utilize a Real-Time Operating System (RTOS). As a result, all software tasks are executed sequentially within the main program loop, which may introduce small delays when multiple operations occur simultaneously, such as UART communication, keypad scanning, Telegram communication, and security monitoring.

8.0 FUTURE WORK AND RECOMMENDATIONS

Although the proposed Automated Parcel Storage and Collection System successfully achieved its intended objectives, several improvements can be implemented to further enhance its functionality, reliability, and scalability.

Firstly, the storage capacity can be increased by expanding the number of storage compartments. This would enable the system to accommodate a larger number of parcels and improve its suitability for deployment in universities, residential buildings, or commercial facilities with higher parcel volumes.

Secondly, an **administrator storage access mode** should be incorporated into future versions of the system. This feature would allow authorized personnel to manually retrieve parcels that have remained uncollected for an extended period. By removing overdue parcels, storage space can be recovered, allowing the system to continue accepting new deliveries and preventing storage congestion.

Furthermore, the software can be enhanced by implementing a **Real-Time Operating System (RTOS)**. An RTOS would enable multiple software tasks, such as keypad scanning, UART communication, Telegram messaging, Wi-Fi management, and security monitoring, to execute concurrently with deterministic scheduling. This would improve system responsiveness, reduce processing delays, and provide better overall performance, particularly if the system is expanded with additional hardware modules or more complex functionalities.

Cloud database integration can also be introduced to store parcel records, collection history, and system logs remotely. This would allow centralized data management, remote monitoring, and easier retrieval of historical parcel information.

Finally, additional authentication methods such as RFID cards, fingerprint recognition, or facial recognition can be integrated alongside the existing PIN verification system to further enhance security. A dedicated mobile application may also be developed to complement the Telegram Bot by providing a more comprehensive and user-friendly interface for parcel tracking, notifications, and system management.

9.0 CONCLUSION

This project successfully designed and developed an **Automated Parcel Storage and Collection System** using two Raspberry Pi Pico microcontrollers to automate the parcel delivery, storage, and collection processes. By integrating embedded systems, automation, secure authentication, and Internet of Things (IoT) technology, the proposed system provides a practical solution for improving parcel management within a university environment. The modular architecture, consisting of separate delivery and storage controllers, enabled efficient task distribution while maintaining reliable communication through one-way UART and a dedicated GPIO signal for storage status monitoring.

The experimental studies conducted during the development process successfully determined the optimum operating parameters for the laser-LDR parcel detection system and the conveyor mechanism. The selected LDR threshold of **400 lux** provided reliable parcel detection under normal indoor lighting conditions, while the conveyor running time of **15 seconds** ensured that parcels were consistently transported to the storage compartment with an additional safety margin to improve operational reliability.

The developed system successfully achieved all the objectives established at the beginning of the project. The parcel delivery process was automated through student identification verification, parcel detection, conveyor control, and communication between the two Raspberry Pi Pico microcontrollers. Secure parcel collection was implemented using randomly generated four-digit PINs, while the integrated Telegram Bot provided real-time notifications, parcel reminders, collection confirmations, and security alerts. The security mechanism also demonstrated reliable performance by activating an alarm after three consecutive incorrect PIN attempts, thereby preventing unauthorized parcel collection.

Overall, the proposed Automated Parcel Storage and Collection System demonstrated reliable performance throughout testing and successfully integrated hardware, software, wireless communication, and security features into a single automated platform. The project demonstrates the practical application of embedded systems and IoT technologies in addressing real-world parcel management challenges. With the future improvements suggested in this report, the system has the potential to be further developed into a scalable and intelligent parcel management solution suitable for broader applications beyond the university environment.

References

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico Series Documentation*. Cambridge, U.K.: Raspberry Pi Ltd. [Online]. Available: <https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf> (Accessed: Jul. 11, 2026).
- [2] Raspberry Pi Ltd., *RP2040 Datasheet*. Cambridge, U.K.: Raspberry Pi Ltd., 2024. [Online]. Available: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf> (Accessed: Jul. 11, 2026).
- [3] Adafruit Industries, *CircuitPython Documentation*. New York, NY, USA: Adafruit Industries. [Online]. Available: <https://docs.circuitpython.org> (Accessed: Jul. 11, 2026).
- [4] Telegram Messenger LLP, *Telegram Bot API*. [Online]. Available: <https://core.telegram.org/bots/api> (Accessed: Jul. 11, 2026).
- [5] S. Monk, *Programming the Raspberry Pi Pico in C*. New York, NY, USA: McGraw-Hill Education, 2021.

You Tube Video Link:

https://youtu.be/w89yXCtu_S8?si=etVViC6-UAvduJDZ

Appendix A:

<https://github.com/malikabuzaid55214-sys/Embedded-Systems-Mini-Project.git>